Git Hands-On Labs — Full Worked Solutions (HOL 1–5)

> All 5 labs build on the same local repo, **GitDemo**, and the same remote (GitLab/GitHub). Run each lab's commands in order in Git Bash. Replace `<your-name>` / `<your-email>` / `<remote-url>` with your own values.

HOL 1 — Git Setup, Notepad++ Integration & First Commit

Objectives: configure Git, set Notepad++ as the default editor, initialize a repo, and push a first file.
Estimated time: 30 minutes

Step 1: Set up Git configuration

1. Verify Git is installed

```
git --version
```

Output such as `git version 2.44.0.windows.1` confirms the Git client is installed correctly.

2. Set your user-level identity (used on every commit)

```
git config --global user.name "<your-name>"
git config --global user.email "<your-email>"
```

3. Verify the configuration

```
git config --global --list
# or check individual values
git config --global user.name
git config --global user.email
```

Step 2: Integrate Notepad++ as the default Git editor

1. Check if Notepad++ is callable from Git Bash

```
notepad++
```

If Git Bash reports `command not found`, `notepad++.exe` is not on the environment `PATH`.

Add it to PATH (Windows): `Control Panel → System → Advanced system settings → Environment Variables → Path → Edit → New` → add the Notepad++ install folder, e.g. `C:\Program`

`Files\Notepad++`.

2. Re-open Git Bash and test again

```
notepad++
```

Notepad++ should now launch from the shell.

3. Create an alias for convenience — add to `~/.bashrc`:

```
echo alias notepad++=\"'/c/Program Files/Notepad++/notepad++.exe'\" >> ~/.bashrc
source ~/.bashrc
```

4. Configure Notepad++ as Git's core editor

```
git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -
multiInst -notabbar -nosession -noPlugin"
```

5. Verify the default editor

```
git config -e
# or
git config --global -e
```

`-e` opens the editor config directly; `git config --global --list` will also show `core.editor=...` in the output.

Step 3: Add a file to the source-code repository

1. Create and initialize the "GitDemo" project

```
mkdir GitDemo
cd GitDemo
git init
```

2. Verify initialization

```
ls -la
```

The hidden `.git/` folder confirms GitDemo is now a Git working directory.

3. Create `welcome.txt` with content

```
echo "Welcome to my first Git repository" > welcome.txt
```

4. Verify the file exists

```
ls -la
```

5. Verify its content

```
cat welcome.txt
```

6. Check status

```
git status
```

`welcome.txt` shows as **untracked** — it exists in the working directory but Git isn't tracking it yet.

7. Stage the file (start tracking it)

```
git add welcome.txt
```

8. Commit with a multi-line message using the default editor

```
git commit
```

Notepad++ opens — type a summary line, a blank line, then a detailed description, save, and close the editor to complete the commit.

9. Confirm the working directory and local repo are in sync

```
git status
```

Should report `nothing to commit, working tree clean` — `welcome.txt` is now committed to the local repository.

10–12. Connect to the remote and sync

```
# Create "GitDemo" project on GitLab first, then:
git remote add origin <remote-url>
git pull origin master --allow-unrelated-histories
git push origin master
```

HOL 2 — `.gitignore`

Objectives: ignore unwanted files/folders using `.gitignore`. **Estimated time:** 20 minutes

1. Create a `.log` file and a `log` folder in the GitDemo working directory

```
cd GitDemo
echo "sample log entry" > app.log
mkdir log
echo "sample nested log" > log/debug.log
```

2. Check status before ignoring anything

```
git status
```

Both `app.log` and `log/` appear as **untracked**.

3. Create/update `.gitignore` to exclude `.log` files and the `log` folder

```
touch .gitignore
echo "*.log" >> .gitignore
echo "log/" >> .gitignore
```

4. Stage and commit `.gitignore` itself (the ignore rules must be tracked)

```
git add .gitignore
git commit -m "Add .gitignore to exclude log files and log folder"
```

5. Verify the ignore rules work

```
git status
```

`app.log` and `log/` no longer appear as untracked — Git is now ignoring them in the working directory, local repository staging, and any future commits.

6. Confirm with an explicit ignore check

```
git check-ignore -v app.log
git check-ignore -v log/debug.log
```

Each command prints the matching `.gitignore` rule and line number, confirming both are correctly excluded.

7. Push the `.gitignore` update to the remote

```
git push origin master
```

HOL 3 — Branching & Merging

Objectives: explain branching/merging, and (conceptually) branch/merge requests in GitLab. **Estimated time:** 30 minutes **Prerequisite:** Git environment configured with **P4Merge** for Windows.

Branching

1. Create a new branch `GitNewBranch`

```
git branch GitNewBranch
```

2. List all local and remote branches

```
git branch -a
```

The `*` prefix marks the branch you are currently on (still `master`/`main` at this point).

3. Switch to the new branch and add files with content

```
git checkout GitNewBranch
echo "Feature work in progress" > feature.txt
```

4. Stage and commit the changes

```
git add feature.txt
git commit -m "Add feature.txt with initial content on GitNewBranch"
```

5. Check status

```
git status
```

Should show a clean working tree on `GitNewBranch`.

Merging

1. Switch back to master

```
git checkout master
```

2. List command-line differences between master and the branch

```
git diff master GitNewBranch
```

3. List visual differences using P4Merge

```
git difftool -t p4merge master GitNewBranch
```

(Requires `difftool.p4merge.cmd` configured in `.gitconfig`, e.g.:

```
git config --global diff.tool p4merge
git config --global difftool.p4merge.cmd "\"C:/Program
Files/Perforce/p4merge.exe\" \"$LOCAL\" \"$REMOTE\""
```)

4. Merge the branch into master
```bash
git merge GitNewBranch
```

5. Observe the log after merging

```
git log --oneline --graph --decorate
```

This shows the commit graph with branch/merge points and current `HEAD`/branch labels.

6. Delete the branch after merging, and check status

```
git branch -d GitNewBranch
git status
git branch -a
```

`-d` only deletes if the branch is fully merged (safe delete); the branch is gone from the local list while its commits remain in `master`'s history.

> **GitLab branch/merge requests (UI equivalent):** push the branch (`git push origin GitNewBranch`), then in GitLab go to **Repository** → **Branches** to create a branch from the UI, and **Merge Requests** → **New merge request** to open a request from `GitNewBranch` into `master`, request review/approval, and merge from the web UI.

HOL 4 — Merge Conflict Resolution

Objectives: resolve conflicts when master and a branch modify the same file differently. **Prerequisite:** HOL 3 (`Git-T03-HOL_001`) completed. **Estimated time:** 30 minutes

1. Verify master is clean

```
git checkout master
git status
```

2. Create branch `GitWork` and add `hello.xml`

```
git checkout -b GitWork
echo "<message>Hello from GitWork branch</message>" > hello.xml
git add hello.xml
git commit -m "Add hello.xml on GitWork"
```

3. Update the content of `hello.xml` and observe status

```
echo "<message>Updated on GitWork branch</message>" > hello.xml
git status
```

Shows `hello.xml` as **modified**.

4. Commit the change on the branch

```
git add hello.xml
git commit -m "Update hello.xml content on GitWork"
```

5. Switch to master

```
git checkout master
```

6. Add a *different* `hello.xml` on master

```
echo "<message>Hello from master branch</message>" > hello.xml
```

7. Commit the change to master

```
git add hello.xml
git commit -m "Add hello.xml on master with different content"
```

8. Observe the diverged history

```
git log --oneline --graph --decorate --all
```

Shows two diverging commit lines — one under `master`, one under `GitWork` — both touching `hello.xml`.

9. Check differences with the Git diff tool

```
git diff master GitWork -- hello.xml
```

10. Visualize differences with P4Merge

```
git difftool -t p4merge master GitWork -- hello.xml
```

11. Merge the branch into master

```
git merge GitWork
```

Git reports a **CONFLICT (content): Merge conflict in hello.xml** because both branches changed the same lines.

12. Observe Git's conflict markup

```
cat hello.xml
```

```
<<<<<<< HEAD
<message>Hello from master branch</message>
=======
<message>Updated on GitWork branch</message>
>>>>>>> GitWork
```

`<<<<<<< HEAD` ... `=======` ... `>>>>>>> GitWork` delimits master's version from the incoming branch's version.

13. Use a 3-way merge tool to resolve it

```
git mergetool -t p4merge
```

P4Merge opens three panes — **local** (master), **remote** (GitWork), and **base** (common ancestor) — plus the merged-result pane. Pick/combine the correct content, save, and close.

14. Commit the resolved merge

```
git add hello.xml
git commit -m "Resolve merge conflict in hello.xml between master and GitWork"
```

15. Check status and ignore the mergetool's backup file

```
git status
```

`hello.xml.orig` (created by the merge tool) shows as untracked.

```
echo "*.orig" >> .gitignore
```

16. Commit the `.gitignore` update

```
git add .gitignore
git commit -m "Ignore .orig backup files created by merge tool"
```

17. List all available branches

```
git branch -a
```

18. Delete the now-merged branch

```
git branch -d GitWork
```

19. Observe the final log

```
git log --oneline --graph --decorate
```

Shows the merge commit joining both histories, with `GitWork` removed from the branch list but its commits preserved in history.

HOL 5 — Clean-Up & Push Back to Remote

Objectives: clean up local state and push pending work to remote Git. **Prerequisite:** HOL 4 (`Git-T03-HOL_002`) completed. **Estimated time:** 10 minutes

1. Verify master is in a clean state

```
git checkout master
git status
```

Should report `nothing to commit, working tree clean`.

2. List all available branches

```
git branch -a
```

Confirms `GitWork` (deleted locally in HOL 4) is gone, and shows what remains locally/remotely.

3. Pull the remote repository into master (bring down anything committed remotely since your last sync)

```
git pull origin master
```

4. Push the pending changes from HOL 4 (`Git-T03-HOL_002`) to the remote

```
git push origin master
```

This publishes the conflict-resolution merge commit, the `.gitignore` update, and any other local commits made during that lab.

5. Verify the changes reflect on the remote

```
git log origin/master --oneline --graph --decorate
```

Or open the GitLab/GitHub project in a browser and confirm the latest commits (merge commit, `.gitignore` update) appear in the file history / commit log.

Quick Reference — Commands Used Across All Labs

Purpose	Command
Configure identity	`git config --global user.name/user.email`
Set default editor	`git config --global core.editor "..."`
Initialize repo	`git init`
Check status	`git status`
Stage changes	`git add <file>`
Commit	`git commit -m "..."` or `git commit` (opens editor)
Connect remote	`git remote add origin <url>`
Sync with remote	`git pull origin master` / `git push origin master`
Ignore files	edit `.gitignore` (`*.log`, `log/`, `*.orig`, etc.)
Create branch	`git branch <name>` or `git checkout -b <name>`
Switch branch	`git checkout <name>`
List branches	`git branch -a`
Compare branches	`git diff <branch1> <branch2>` / `git difftool -t p4merge ...`
Merge	`git merge <branch>`
Resolve conflicts	`git mergetool -t p4merge`, then `git add` + `git commit`
View history	`git log --oneline --graph --decorate --all`
Delete branch	`git branch -d <name>`

Submission Checklist

- [ ] All 5 labs completed against the same **GitDemo** repo
- [ ] `.gitignore` correctly excludes `*.log`, `log/`, and `*.orig`
- [ ] Merge conflict in `hello.xml` resolved and committed
- [ ] Final `git log --oneline --graph --decorate` matches expected history
- [ ] All commits pushed and visible on the remote (GitLab/GitHub)